

UNIT – I

INTRODUCTION AND ANALYSIS OF ALGORITHMS

INTRODUCTION

- **What is an Algorithm ?**
- An algorithm is a finite set of instructions or logic, written in order, to accomplish a certain predefined task.
- Algorithm is not the complete code or program, it is just the core logic(solution) of a problem, which can be expressed either as an informal high level description as **pseudocode** or using a **flowchart**.
- Every Algorithm must satisfy the following properties:
- **Input**- There should be 0 or more inputs supplied externally to the algorithm.
- **Output**- There should be atleast 1 output obtained.
- **Definiteness**- Every step of the algorithm should be clear and well defined.
- **Finiteness**- The algorithm should have finite number of steps.
- **Correctness**- Every step of the algorithm must generate a correct output.

CHARACTERISTICS OF ALGORITHMS

- ☐ Clear and Unambiguous: Algorithm should be clear and unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
- ☐ Well-Defined Inputs: If an algorithm says to take inputs, it should be well defined inputs.
- ☐ Well-Defined Outputs: The algorithm must clearly define what output will be yielded and it should be well-defined as well.
- ☐ Finite-ness: The algorithm must be finite, i.e. it should not end up in an infinite loops or similar.
- ☐ Feasible: The algorithm must be simple, generic and practical, such that it can be executed upon will the available resources. It must not contain some future technology, or anything.
- ☐ Language Independent: The Algorithm designed must be language independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be same, as expected.

PROS AND CONS

- Advantages of Algorithms:

- ☐ It is easy to understand.
- ☐ Algorithm is a step-wise representation of a solution to a given problem.
- ☐ In Algorithm the problem is broken down into smaller pieces or steps hence, it is easier for the programmer to convert it into an actual program.

- Disadvantages of Algorithms:

- ☐ Writing an algorithm takes a long time so it is time-consuming.
- ☐ Branching and Looping statements are difficult to show in Algorithms.

ALGORITHM TO ADD TWO NUMBERS AND DISPLAY THE RESULT.

- Step 1 – START
- Step 2 – declare three integers a, b & c
- Step 3 – define values of a & b
- Step 4 – add values of a & b
- Step 5 – store output of step 4 to c
- Step 6 – print c
- Step 7 – STOP

ALGORITHM COMPLEXITY

- An algorithm is said to be efficient and fast, if it takes less time to execute and consumes less memory space. The performance of an algorithm is measured on the basis of following properties :
- Suppose X is an algorithm and n is the size of input data, the time and space used by the algorithm X are the two main factors, which decide the efficiency of X .
- ☐ Time Factor — Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.
- ☐ Space Factor — Space is measured by counting the maximum memory space required by the algorithm.
- The complexity of an algorithm $f(n)$ gives the running time and/or the storage space required by the algorithm in terms of n as the size of input data.

- Algorithms tell the programmers how to code the program. Alternatively, the algorithm can be written as —
- Step 1 — START ADD
- Step 2 — get values of a & b
- Step 3 — $c \leftarrow a + b$
- Step 4 — display c
- Step 5 — STOP

SPACE COMPLEXITY

- Space Complexity of an algorithm represents the amount of memory space required by the algorithm in its life cycle. The space required by an algorithm is equal to the sum of the following two components
 - A fixed part that is a space required to store certain data and variables, that are independent of the size of the problem. For example, simple variables and constants used, program size, etc.
 - A variable part is a space required by variables, whose size depends on the size of the problem. For example, dynamic memory allocation, recursion stack space, etc.
- Space complexity $S(P)$ of any algorithm P is $S(P) = C + SP(I)$, where C is the fixed part and $S(I)$ is the variable part of the algorithm, which depends on instance characteristic I . Following is a simple example that tries to explain the concept —
- Algorithm:
 - SUM(A, B)
 - Step 1 - START
 - Step 2 - $C \leftarrow A + B + 10$
 - Step 3 - Stop
- Here we have three variables A, B, and C and one constant.
- Hence $S(P) = 1 + 3$.
- Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

TIME COMPLEXITY

- Time complexity of an algorithm represents the amount of time required by the algorithm to run to completion. Time requirements can be defined as a numerical function $T(n)$, where $T(n)$ can be measured as the number of steps, provided each step consumes constant time.
- For example, addition of two n -bit integers takes n steps.
- Consequently, the total computational time is $T(n) = c * n$, where c is the time taken for the addition of two bits. Here, we observe that $T(n)$ grows linearly as the input size increases.

ALGORITHM ANALYSIS

- Efficiency of an algorithm can be analyzed at two different stages, before implementation and after implementation.
- ☐ A Priori Analysis or Performance or Asymptotic Analysis – This is a theoretical analysis of an algorithm.
- Efficiency of an algorithm is measured by assuming that all other factors,
- for example, processor speed, are constant and have no effect on the implementation.
- ☐ A Posterior Analysis or Performance Measurement – This is an empirical analysis of an algorithm. The selected algorithm is implemented using programming language.
- This is then executed on target computer machine. In this analysis, actual statistics like running time and space required, are collected.

ANALYSIS TYPES

- **Types of Analysis** The efficiency of some algorithms may vary for inputs of the same size. For such algorithms, we need to differentiate between the worst case, average case and best case efficiencies.
 - BEST CASE
 - WORST CASE
 - AVERAGE CASE
- **Best Case Analysis** If an algorithm takes the least amount of time to execute a specific set of input, then it is called the best case time complexity. The best case efficiency of an algorithm is the efficiency for the best case input of size n . Because of this input, the algorithm runs the fastest among all the possible inputs of the same size.

- **Average Case Analysis** If the time complexity of an algorithm for certain sets of inputs are on an average, then such a time complexity is called average case time complexity. Average case analysis provides necessary information about an algorithm's behavior on a typical or random input. You must make some assumption about the possible inputs of size n to analyze the average case efficiency of algorithm.
- **Worst Case Analysis** If an algorithm takes maximum amount of time to execute for a specific set of input, then it is called the worst case time complexity. The worst case efficiency of an algorithm is the efficiency for the worst case input of size n . The algorithm runs the longest among all the possible inputs of the similar size because of this input of size n .

ASYMPTOTIC NOTATIONS

- choose the best algorithm for a particular process
- you must be able to judge the time taken to run a particular solution.
- More accurately, you must be able to judge the time taken to run two solutions, and choose the better among the two.
- To select the best algorithm, it is necessary to check the efficiency of each algorithm.
- The efficiency of each algorithm can be checked by computing its time complexity.
- The asymptotic notations help to represent the time complexity in a shorthand way.
- It can generally be represented as the fastest possible, slowest possible or average possible.

NOTATIONS - TIME COMPLEXITY

- O (Big-O)
- Ω (Omega)
- θ (Theta)

BIG-O NOTATION

- 'O' is the representation for Big-O notation. Big -O is the method used to express the upper bound of the running time of an algorithm.
- It is used to describe the performance or time complexity of the algorithm.
- Big-O specifically describes the worst-case scenario and can be used to describe the execution time required or the space used by the algorithm

- Big-O is defined as: $f(n) \leq c * g(n)$
- n can be any number of inputs or outputs and $f(n)$ as well as $g(n)$ are two non-negative functions.
- These functions are true only if there is a constant c and a non-negative integer n_0 such that, $n \geq n_0$.
- The Big-O can also be denoted as $f(n) = O(g(n))$, where $f(n)$ and $g(n)$ are two non-negative functions and
- $f(n) < g(n)$ if $g(n)$ is multiple of some constant c .
- The graphical representation of $f(n) = O(g(n))$, where the running time increases considerably when n increases.
- Example :
- Consider function $f(n) = 2(n)+2$ and $g(n) = n^2$.

COMMON ORDERS

- $O(1)$ Constant Adding to the front of a linked list
- $O(\log n)$ Logarithmic Finding an entry in a sorted array
- $O(n)$ Linear Finding an entry in an unsorted array
- $O(n \log n)$ Linearithmic Sorting 'n' items by 'divide-and-conquer'
- $O(n^2)$ Quadratic Shortest path between two nodes in a graph
- $O(n^3)$ Cubic Simultaneous linear equations
- $O(2^n)$ Exponential The Towers of Hanoi problem

EXAMPLE

- Consider function $f(n) = 2(n)+2$ and $g(n) = n^2$.
- We need to find the constant c such that
- $f(n) \leq c * g(n)$.
- Let $n = 1$, then
- $f(n) = 2(n)+2 = 2(1)+2 = 4$
- $g(n) = n^2 = 1^2 = 1$
- Here, $f(n) > g(n)$
- Let $n = 2$, then
- $f(n) = 2(n)+2 = 2(2)+2 = 6$
- $g(n) = n^2 = 2^2 = 4$
- Here, $f(n) > g(n)$
- Let $n = 3$, then $f(n) = 2(n)+2 = 2(3)+2 = 8$
- $g(n) = n^2 = 3^2 = 9$
- Here, $f(n) < g(n)$

EXAMPLE

- Example:
- Consider $f(n) = 15n^3 + 40n^2 + 2n \log n + 2n$.
- As the value of n increases, n^3 becomes much larger than n^2 , $n \log n$, and n .
- Hence, it dominates the function $f(n)$ and we can consider the running time to grow by the order of n^3 .
- Therefore, it can be written as $f(n) = O(n^3)$. The values of n for $f(n)$ and $C^* g(n)$ will not be less than n_0 . Therefore, the values less than n_0 are not considered relevant.

- Thus, when n is greater than 2, we get $f(n) < g(n)$. In other words, as n becomes larger, the running time increases considerably. This concludes that the Big-O helps to determine the 'upper bound' of the algorithm's run-time.

OMEGA NOTATION

- 'Ω' is the representation for Omega notation. Omega describes the manner in which an algorithm performs in the best case time complexity. This notation provides the minimum amount of time taken by an algorithm to compute a problem. Thus, it is considered that omega gives the "lower bound" of the algorithm's run-time.
- Omega is defined as: $f(n) \geq c * g(n)$
- Where, n is any number of inputs or outputs and f(n) and g(n) are two non-negative functions. These functions are true only if there is a constant c and a non-negative integer n₀ such that $n > n_0$.
- Omega can also be denoted as $f(n) = \Omega(g(n))$ where, f of n is equal to Omega of g of n. The graphical representation of $f(n) = \Omega(g(n))$. The function f(n) is said to be in $\Omega(g(n))$, if f(n) is bounded below by some constant multiple of g(n) for all large values of n, i.e., if there exists some positive constant c and some non-negative integer n₀, such that $f(n) \geq c * g(n)$ for all $n \geq n_0$.

EXAMPLE

- Consider function $f(n) = 2n^2+5$ and $g(n) = 7n$.
- We need to find the constant c such that $f(n) \geq c * g(n)$.
- Let $n = 0$, then
 - $f(n) = 2n^2+5 = 2(0)^2+5 = 5$
 - $g(n) = 7(n) = 7(0) = 0$
- Here, $f(n) > g(n)$
- Let $n = 1$, then
 - $f(n) = 2n^2+5 = 2(1)^2+5 = 7$
 - $g(n) = 7(n) = 7(1) = 7$
- Here, $f(n) = g(n)$
- Let $n = 2$, then
 - $f(n) = 2n^2+5 = 2(2)^2+5 = 13$
 - $g(n) = 7(n) = 7(2) = 14$
- Here, $f(n) < g(n)$

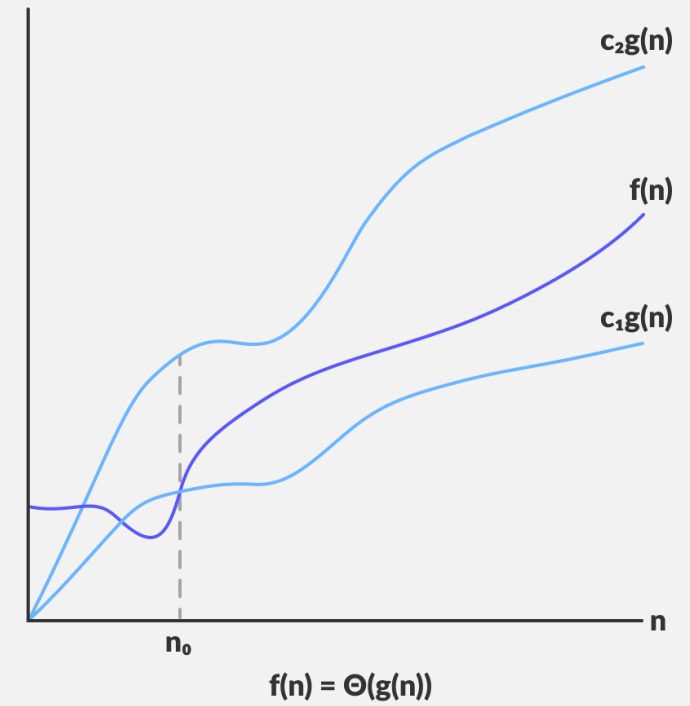
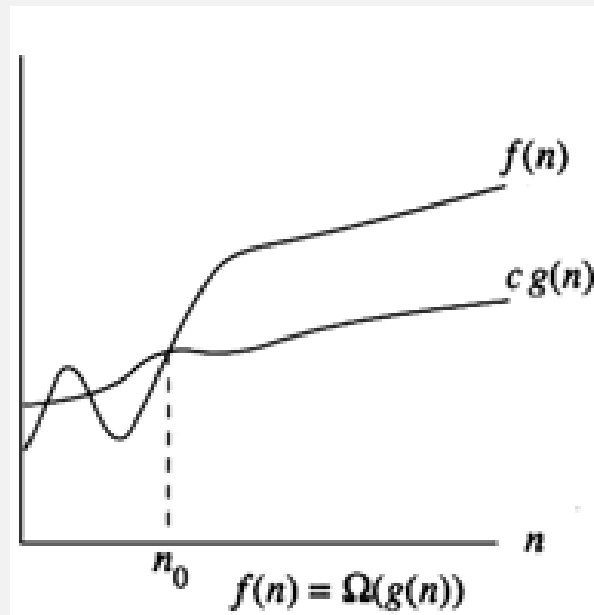
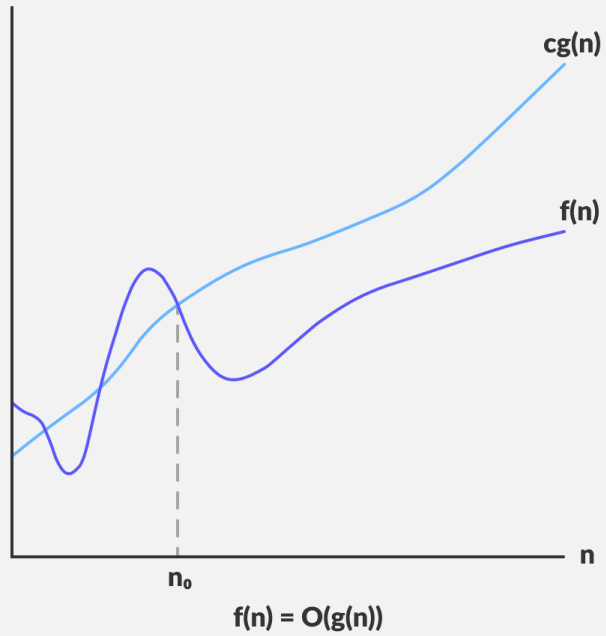
THETA NOTATION

- 'θ' is the representation for Theta notation. Theta notation is used when the upper bound and lower bound of an algorithm are in the same order of magnitude.
- Theta can be defined as: $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all $n > n_0$
- Where, n is any number of inputs or outputs and $f(n)$ and $g(n)$ are two nonnegative functions. These functions are true only if there are two constants namely, c_1 , c_2 , and a non-negative integer n_0 .
- Theta can also be denoted as $f(n) = \theta(g(n))$ where, f of n is equal to Theta of g of n . The graphical representation of $f(n) = \theta(g(n))$ is shown in figure 2.3. The function $f(n)$ is said to be in $\theta(g(n))$ if $f(n)$ is bounded both above and below by some positive constant multiples of $g(n)$ for all large values of n , i.e., if there exists some positive constant c_1 and c_2 and some non-negative integer n_0 , such that $C_2g(n) \leq f(n) \leq C_1g(n)$ for all $n \geq n_0$.

EXAMPLE

- Consider function $f(n) = 4n + 3$ and $g(n) = 4n$ for all $n \geq 3$; and $f(n) = 4n + 3$ and $g(n) = 5n$ for all $n \geq 3$.
- Then the result of the function will be:
- Let $n = 3$
- $f(n) = 4n + 3 = 4(3) + 3 = 15$
- $g(n) = 4n = 4(3) = 12$ and
- $f(n) = 4n + 3 = 4(3) + 3 = 15$
- $g(n) = 5n = 5(3) = 15$ and here, c_1 is 4, c_2 is 5 and n_0 is 3
- Thus, from the above equation we get $c_1 g(n) \leq f(n) \leq c_2 g(n)$. This concludes that Theta notation depicts the running time between the upper bound and lower bound.

GRAPHICAL REPRESENTATION



LOOP INVARIANT

A loop invariant is a condition [among program variables] that is necessarily true immediately before and immediately after each iteration of a loop. (Note that this says nothing about its truth or falsity part way through an iteration.)

A loop invariant is some predicate (condition) that holds for every iteration of the loop.

For example:

```
int j = 9;  
for(int i=0; i<10; i++)  
j--;
```

The loop invariant must be true:

- before the loop starts
- before each iteration of the loop
- after the loop terminates

(although it can temporarily be false during the body of the loop).

On the other hand the loop conditional must be false after the loop terminates, otherwise, the loop would never terminate.

Loop invariants can be used to prove the correctness of an algorithm, debug an existing algorithm without even tracing the code or develop an algorithm directly from specification.

➤ A good loop invariant should satisfy three properties:

* Initialization: The loop invariant must be true before the first execution of the loop.

* Maintenance: If the invariant is true before an iteration of the loop, it should be true also after the iteration.

* Termination: When the loop is terminated the invariant should tell us something useful, something that helps us understand the algorithm.

➤ Loop invariant condition is a condition about the relationship between the variables of our program which is definitely true immediately before and immediately after each iteration of the loop.

For example: Consider an array $A\{7, 5, 3, 10, 2, 6\}$ with 6 elements and we have to find maximum element $\max$ in the array.

```
max = -INF (minus infinite)
```

```
for (i = 0 to n-1)
```

```
    if ( $A[i] > \max$ )
```

```
        max =  $A[i]$ 
```

In the above example after the 3rd iteration of the loop $\max$ value is 7, which holds true for the first 3 elements of array A. Here, the loop invariant condition is that $\max$ is always maximum among the first i elements of array A.

RECURRENCES- SUBSTITUTION METHOD

➤ Recurrence Relation: Whenever any function makes a recursive call to itself, its time can be computed by a Recurrence Relation. Recurrence Relation is simply a mathematical relation/equation that can give the value of any term in terms of some previous smaller terms. For example,

➤ $T(n) = T(n-1) + N$

>To calculate time we need to solve the recurrence relation. Now for solving recurrence we have three famous methods-

- Substitution Method
- Recursive Tree Method
- Master Theorem

Substitution Method is very famous method for solving any recurrences. There are two types of substitution methods-

- Forward Substitution
- Backward Substitution

➤ I. Forward Substitution:

It is called Forward Substitution because here we substitute recurrence of any term into next terms. It uses following steps to find Time using recurrences-

Pick Recurrence Relation and the given initial Condition

- * Put the value from previous recurrence into the next recurrence
- * Observe and Guess the pattern and the time
- * Prove that the guessed result is correct using mathematical Induction.
- * Now we will use these steps to solve a problem. The problem is-

$$T(n) = T(n-1) + n, n > 1$$

$$T(n) = 1, n = 1$$

- Now we will go step by step-

1. Pick Recurrence and the given initial Condition:

- $T(n)=T(n-1)+n, n>1$
 $T(n)=1, n=1$

2. Put the value from previous recurrence into the next recurrence:

- $T(1) = 1$
 $T(2) = T(1) + 2 = 1 + 2 = 3$
 $T(3) = T(2) + 3 = 1 + 2 + 3 = 6$
 $T(4) = T(3) + 4 = 1 + 2 + 3 + 4 = 10$

3. Observe and Guess the pattern and the time:

- So guessed pattern will be- $T(n) = 1 + 2 + 3 \dots + n = (n * (n+1))/2$ Time Complexity will be $O(n^2)$

4. Prove that the guessed result is correct using mathematical Induction:

- Prove $T(1)$ is true:
- $T(1) = 1 * (1+1)/2 = 2/2 = 1$ and from definition of recurrence we know $T(1) = 1$. Hence proved $T(1)$ is true
- Assume $T(N-1)$ to be true:
- Assume $T(N-1) = ((N - 1) * (N-1+1))/2 = (N * (N-1))/2$ to be true
- Then prove $T(N)$ will be true: $T(N) = T(N-1) + N$ from recurrence definition

Now, $T(N-1) = N * (N-1)/2$ So, $T(N) = T(N-1) + N = (N * (N-1))/2 + N = (N * (N-1) + 2N)/2 = N * (N+1)/2$ And from our

guess also $T(N)=N(N+1)/2$ Hence $T(N)$ is true. Therefore our guess was correct and time will be $O(N^2)$

BACKWARD SUBSTITUTION

- It is called Backward Substitution because here we substitute recurrence of any term into previous terms. It uses following steps to find Time using recurrences-
- Take the main recurrence and try to write recurrences of previous terms
- Take just previous recurrence and substitute into main recurrence
- Again take one more previous recurrence and substitute into main recurrence
- Do this process until you reach to the initial condition
- After this substitute the the value from initial condition and get the solution

- The problem is-
- $T(n) = T(n-1) + n, n > 1$ $T(n) = 1, n = 1$

Now we will go step by step-

1. Take the main recurrence and try to write recurrences of previous terms:

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + n - 1$$

$$T(n-2) = T(n-3) + n - 2$$

2. Take just previous recurrence and substitute into main recurrence

put $T(n-1)$ into $T(n)$ So, $T(n) = T(n-2) + n - 1 + n$

3. Again take one more previous recurrence and substitute into main recurrence

put $T(n-2)$ into $T(n)$ So, $T(n) = T(n-3) + n - 2 + n - 1 + n$

4. Do this process until you reach to the initial condition

So similarly we can find $T(n-3), T(n-4), \dots$ and so on and can insert into $T(n)$. Eventually we will get following: $T(n) = T(1) + 2 + 3 + 4 + \dots + n - 1 + n$

5. After this substitute the value from initial condition and get the solution

Put $T(1) = 1$, $T(n) = 1 + 2 + 3 + 4 + \dots + n - 1 + n = \frac{n(n+1)}{2}$. So Time will be $O(N^2)$

LIMITATIONS OF SUBSTITUTION METHOD:

- It is not guaranteed that we will find the solution as substitution method is based on guesses.
- It doesn't provide guidance on how to make an accurate guess, often relying on intuition or trial and error.
- It may only yield a specific or approximate solution rather than the most general or precise one.

RECURSION TREE METHOD

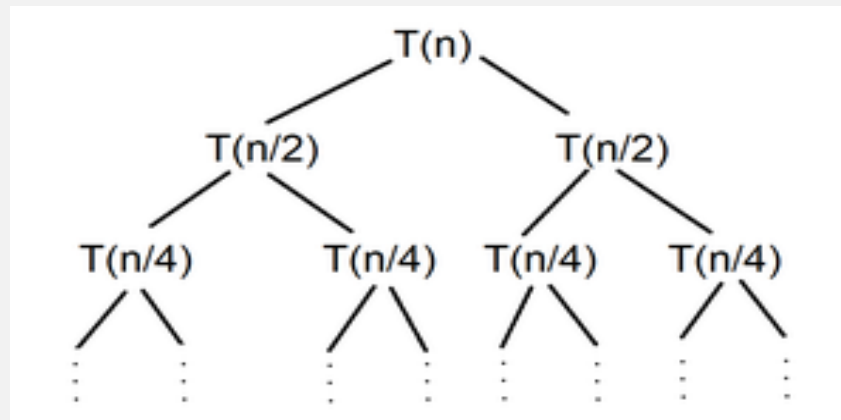
The Recursion Tree Method is a way of solving recurrence relations. In this method, a recurrence relation is converted into recursive trees. Each node represents the cost incurred at various levels of recursion. To find the total cost, costs of all levels are summed up.

> Steps to solve recurrence relation using recursion tree method:

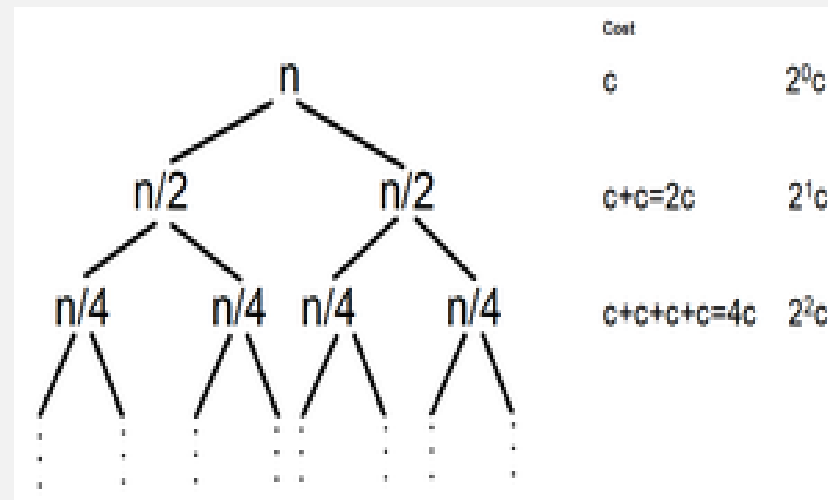
- Draw a recursive tree for given recurrence relation
- Calculate the cost at each level and count the total no of levels in the recursion tree.
- Count the total number of nodes in the last level and calculate the cost of the last level
- Sum up the cost of all the levels in the recursive tree

Examples:

- Question 1:
- $T(n) = 2T(n/2) + c$
- Solution:
- Step 1: Draw a recursive tree



- Step 2: Calculate the work done or cost at each level and count total no of levels in recursion tree



Count the total number of levels -

Choose the longest path from root node to leaf node

$$n/2^0 \rightarrow n/2^1 \rightarrow n/2^2 \rightarrow \dots \rightarrow n/2^k$$

Size of problem at last level = $n/2^k$

At last level size of problem becomes 1

$$n/2^k = 1$$

$$2^k = n$$

$$k = \log_2(n)$$

Total no of levels in recursive tree = $k + 1 = \log_2(n) + 1$

- Step 3: Count total number of nodes in the last level and calculate cost of last level
- No. of nodes at level 0 = $2^0 = 1$
- No. of nodes at level 1 = $2^1 = 2$
-
- No. of nodes at the $(k+1)$ th level (or last level) = $2^k = 2^{\log_2(n)} = n \log_2(2) = n$
- Cost of sub problems at level last level = $n \times \Theta(1) = \Theta(n)$

Step 4: Sum up the cost all the levels in recursive tree

$$\begin{aligned}
 T(n) &= c + 2c + 4c + \dots ((\text{no. of levels}-1) \text{ times}) + \text{last level cost} \\
 &= c + 2c + 4c + \dots (\log_2(n) \text{ times}) + \Theta(n) \\
 &= c(1 + 2 + 4 + \dots (\log_2(n) \text{ times})) + \Theta(n) \\
 1 + 2 + 4 + \dots (\log_2(n) \text{ times}) &\rightarrow 2^0 + 2^1 + 2^2 + \dots (\log_2(n) \text{ times}) \rightarrow \text{Geometric Progression(G.P.)}
 \end{aligned}$$

Using GP Sum Formula, we get the sum of the series as n

$$= c(n) + \Theta(n)$$

$$\text{Thus, } T(n) = \Theta(n)$$

MASTER METHOD

- The Master Theorem is a tool used to solve recurrence relations that arise in the analysis of divide-and-conquer algorithms. The Master Theorem provides a systematic way of solving recurrence relations of the form:
- $T(n) = aT(n/b) + f(n)$
- where a , b , and $f(n)$ are positive functions and n is the size of the problem. The Master Theorem provides conditions for the solution of the recurrence to be in the form of $O(n^k)$ for some constant k , and it gives a formula for determining the value of k .
- Master Theorem is used to determine running time of algorithms (divide and conquer algorithms) in terms of asymptotic notations.

- Consider a problem that is solved using recursion.
 - function $f(\text{input } x \text{ size } n)$
 - if $(n < k)$
 - solve x directly and return
 - else
 - divide x into a subproblems of size n/b
 - call f recursively to solve each subproblem
 - Combine the results of all sub-problems
-
- The above algorithm divides the problem into a subproblems, each of size n/b and solve them recursively to compute the problem and the extra work done for problem is given by $f(n)$, i.e., the time to create the subproblems and combine their results in the above procedure.

- So, according to master theorem the runtime of the above algorithm can be expressed as:

- $T(n) = aT(n/b) + f(n)$

-

where n = size of the problem

a = number of subproblems in the recursion and $a \geq 1$

n/b = size of each subproblem

$f(n)$ = cost of work done outside the recursive calls like dividing into subproblems and cost of combining them to get the solution.

Not all recurrence relations can be solved with the use of the master theorem i.e. if

- $T(n)$ is not monotone, ex: $T(n) = \sin n$
- $f(n)$ is not a polynomial, ex: $T(n) = 2T(n/2) + 2n$

- This theorem is an advance version of master theorem that can be used to determine running time of divide and conquer algorithms if the recurrence is of the following form :-

$$T(n) = aT(n/b) + \theta(n^k \log^p n)$$

where n = size of the problem

a = number of subproblems in the recursion

and $a \geq 1$

n/b = size of each subproblem

$b > 1$, $k \geq 0$ and p is a real number.

Then,

- if $a > b^k$, then $T(n) = \theta(n \log b^a)$
- if $a = b^k$, then
 - (a) if $p > -1$, then $T(n) = \theta(n \log b^a \log p + \ln n)$
 - (b) if $p = -1$, then $T(n) = \theta(n \log b^a \log \log n)$
 - (c) if $p < -1$, then $T(n) = \theta(n \log b^a)$
- if $a < b^k$, then
 - (a) if $p \geq 0$, then $T(n) = \theta(n^k \log p^n)$
 - (b) if $p < 0$, then $T(n) = \theta(n^k)$
- Time Complexity Analysis -
- Example-1: Binary Search - $T(n) = T(n/2) + O(1)$
 $a = 1, b = 2, k = 0$ and $p = 0$
 $b^k = 1$. So, $a = b^k$ and $p > -1$ [Case 2.(a)]
- $T(n) = \theta(n \log b^a \log p + \ln n)$
- $T(n) = \theta(\log n)$

SORTING ALGORITHMS

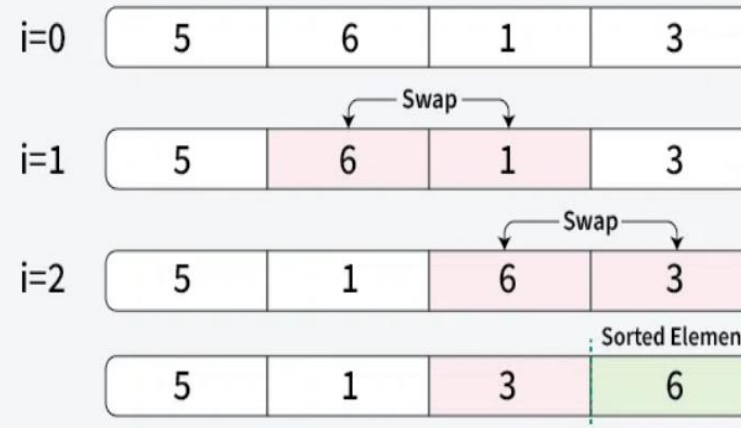
- A Sorting Algorithm is used to rearrange a given array or list of elements in an order. For example, a given array [10, 20, 5, 2] becomes [2, 5, 10, 20] after sorting in increasing order and becomes [20, 10, 5, 2] after sorting in decreasing order.
- Applications:-
 - * Quickly Finding k-th Smallest or K-th Largest
 - * Searching Algorithms.
 - * Data management.
 - * Database optimization
 - * Data Analysis.
 - * Operating Systems.

BUBBLE SORT

- Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity are quite high.

01
Step

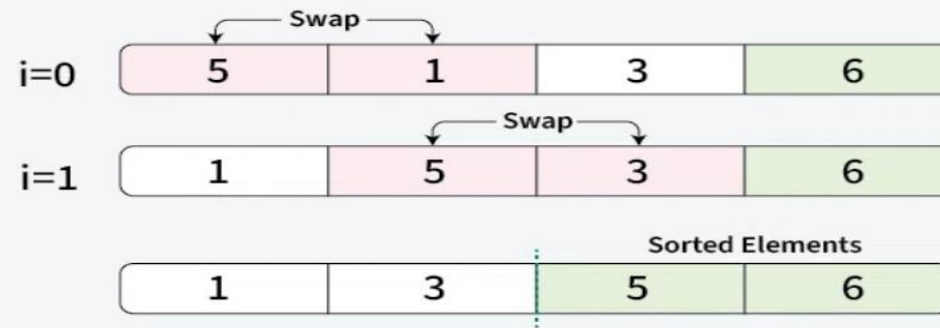
Placing the 1st largest element at its correct position



Bubble sort

02 Step

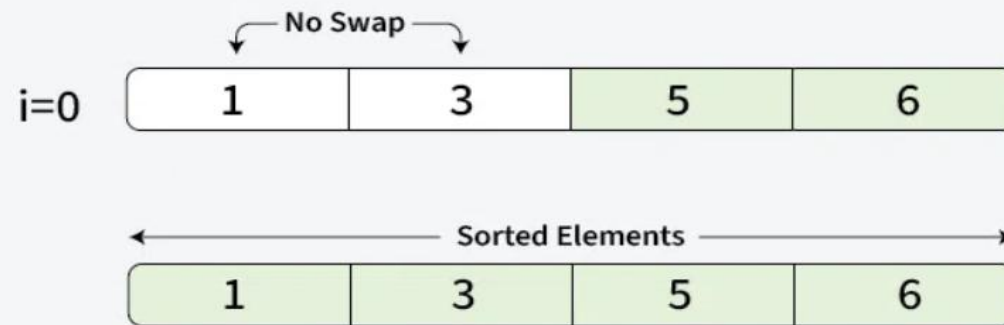
Placing 2nd largest element at its correct position



Bubble sort

03 Step

Placing 3rd largest element at its correct position



Bubble sort

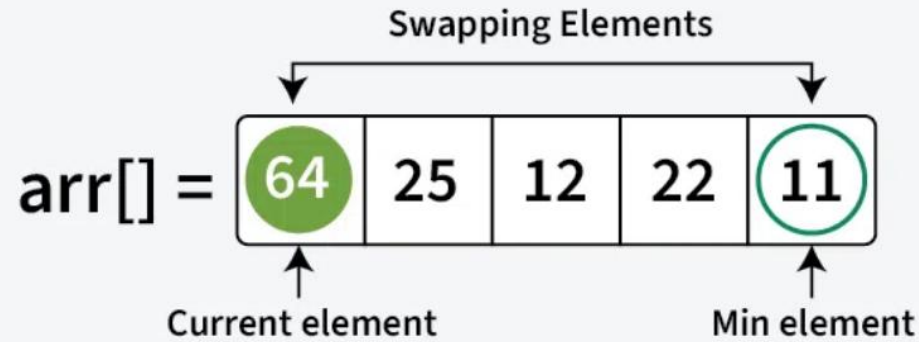
- Advantages of Bubble Sort:
 - Bubble sort is easy to understand and implement.
 - It does not require any additional memory space.
 - It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.
- Disadvantages of Bubble Sort:
 - Bubble sort has a time complexity of $O(n^2)$ which makes it very slow for large data sets.
 - Bubble sort has almost no or limited real world applications. It is mostly used in academics to teach different ways of sorting.

SELECTION SORT

- Selection Sort is a comparison-based sorting algorithm. It sorts an array by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first unsorted element. This process continues until the entire array is sorted.
- First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.
- Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.
- We keep doing this until we get all elements moved to correct position.

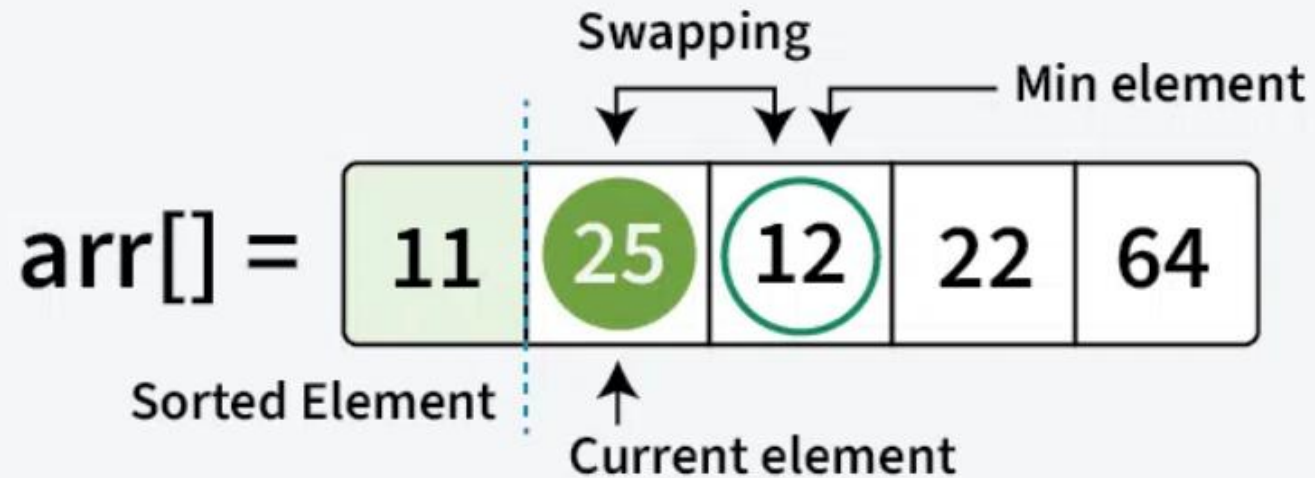
01
Step

Start from the first element at index 0, find the smallest element in the rest of the array which is unsorted, and swap (11) with current element (64).



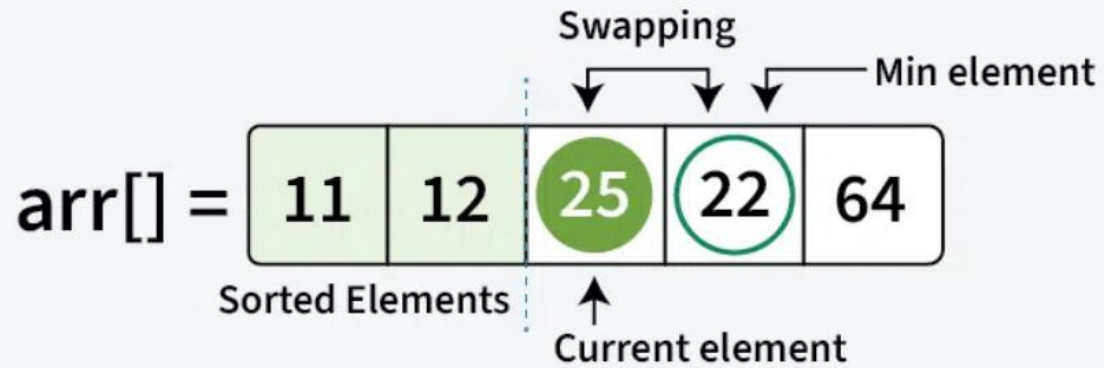
02
Step

Move to the next element at index 1 (25). Find the smallest in unsorted subarray, and swap (12) with current element (25).

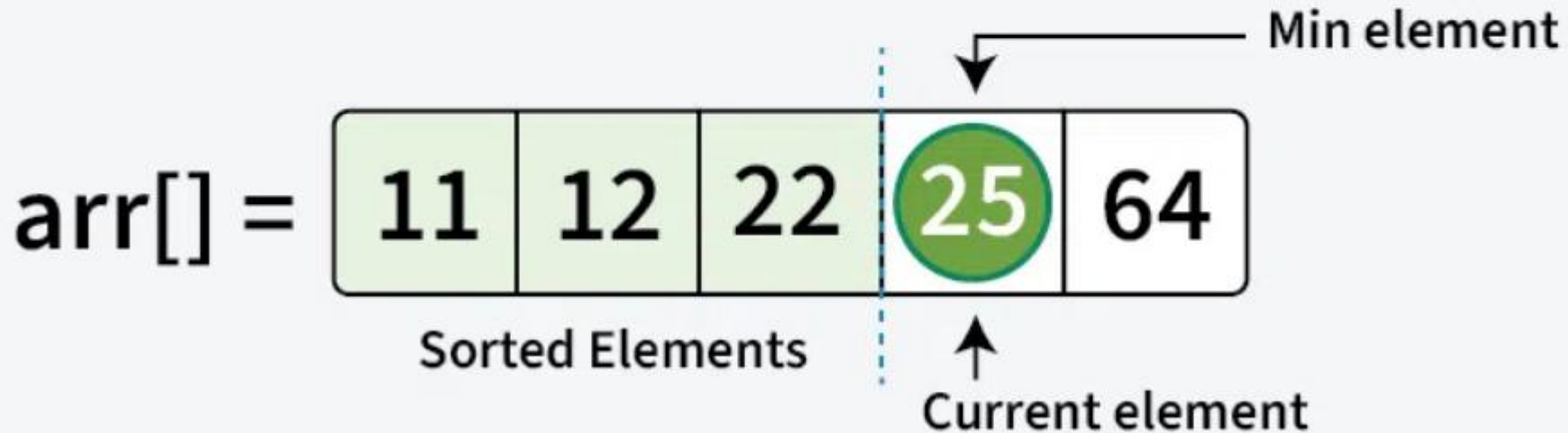


03
Step

Move to element at index 2 (25). Find the minimum element from unsorted subarray, Swap (22) with current element (25).

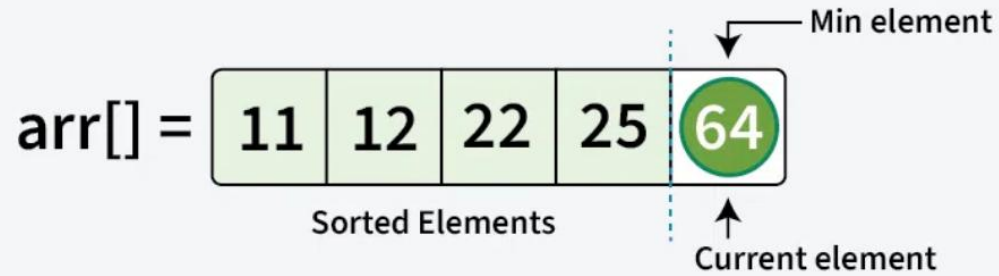
**04**
Step

Move to element at index 3 (25), find the minimum from unsorted subarray and swap (25) with current element (25).



05
Step

Move to element at index 4 (64), find the minimum from unsorted subarray and swap (64) with current element (64).



06
Step

We get the sorted array at the end.



- Advantages of Selection Sort
 - Easy to understand and implement, making it ideal for teaching basic sorting concepts.
 - Requires only a constant $O(1)$ extra memory space.
- Disadvantages of the Selection Sort
 - Selection sort has a time complexity of $O(n^2)$ makes it slower compared to algorithms like Quick Sort or Merge Sort.
- Applications of Selection Sort
 - Perfect for teaching fundamental sorting mechanisms and algorithm design.
 - Heap Sort algorithm is based on Selection Sort.

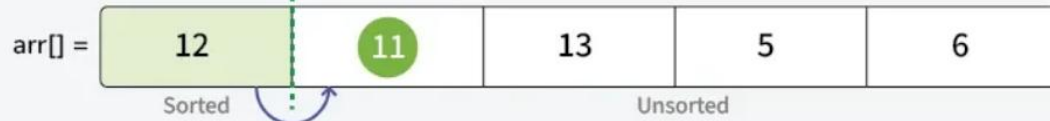
INSERTION SORT

- Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands. You split the cards into two groups: the sorted cards and the unsorted cards. Then, you pick a card from the unsorted group and put it in the right place in the sorted group.
- We start with the second element of the array as the first element is assumed to be sorted.
- Compare the second element with the first element if the second element is smaller then swap them.
- Move to the third element, compare it with the first two elements, and put it in its correct position
- Repeat until the entire array is sorted.

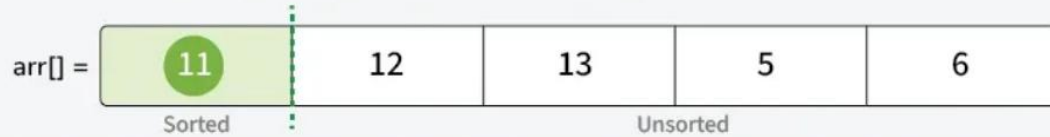
1st Pass

Key = 11

Compare key with 12. As 12 is in the wrong order, shift 12 to the right to make space for the key.



No element left to compare so insert the key at the beginning.



2nd Pass

Key = 13

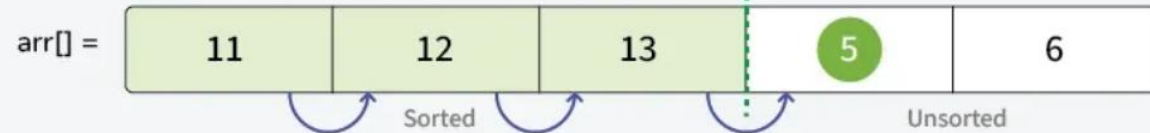
Compare key with 12, 12 is in the correct order, so no changes take place.



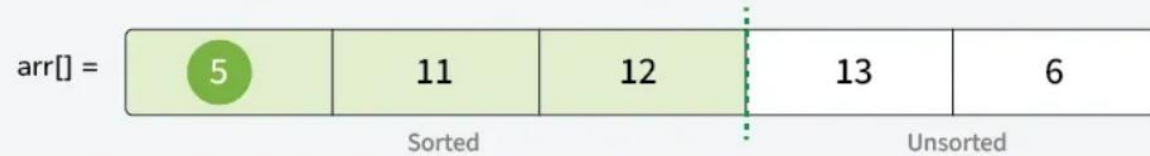
3rd Pass

Key = 5

Compare key with all the elements in the sorted subarray starting with 13, if the element is in the wrong order, shift that element to the right.



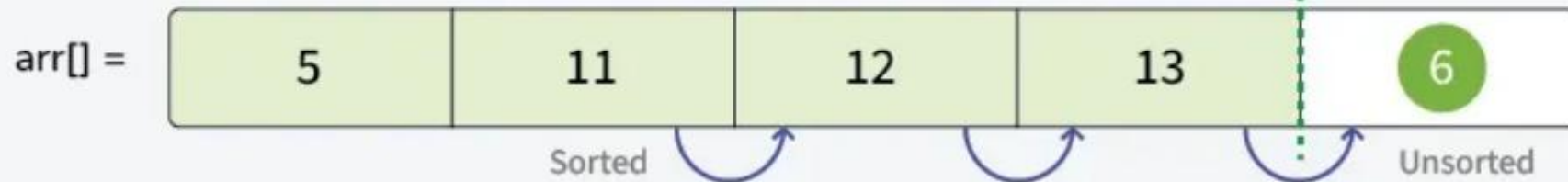
No element left to compare, so insert key at the beginning.



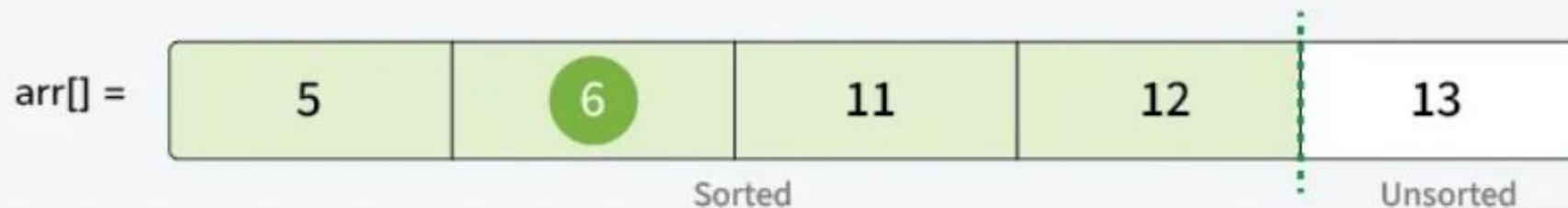
4th Pass

Key = 6

Compare key with all the elements in the sorted subarray starting with 13, if the element is in the wrong order, shift that element to the right.



5 is in the correct order, so shifting stops. Insert key after 5.



Sorted Array

The sorted part contains the whole array. Means that the whole array is sorted.

arr[] =

5	6	11	12	13
---	---	----	----	----

Sorted

- Advantages and Disadvantages of Insertion Sort
- Advantages
 - Simple and easy to implement.
 - Stable sorting algorithm.
 - Efficient for small lists and nearly sorted lists.
 - Space-efficient as it is an in-place algorithm.
- Disadvantages
 - Inefficient for large lists.
 - Not as efficient as other sorting algorithms (e.g., merge sort, quick sort) for most cases.
- Applications of Insertion Sort
 - Insertion sort is commonly used in situations where:
 - The list is small or nearly sorted.
 - Simplicity and stability are important.
 - Used as a subroutine in Bucket Sort
 - Can be useful when array is already almost sorted (very few inversions)